

EXCEL AI ANALYZER

PROJECT DOCUMENTATION

Automated AI-powered analysis of Excel and CSV data
via natural language queries

Web application / Python / Flask / Multi-API AI with auto-fallback

Excel AI Analyzer Project Documentation | Page 1 of 10

1. EXECUTIVE SUMMARY

Project Name

Excel AI Analyzer

Objective

To enable non-technical users to analyse Excel and CSV data using plain-English or voice queries, replacing the slow, error-prone task of writing formulas, pivot tables, or code.

The solution aims to:

Eliminate manual effort Eliminate the need to write formulas, filters, or scripts to extract insights from spreadsheet data

Improve accuracy Let AI interpret the query and return precise, structured results with column-aware reasoning

Improve speed Process multiple files simultaneously with sub-second AI responses and instant local fallback

Improve traceability Produce a structured JSON result with analysis summary, insights, and per-file tables for every query

Stay accessible Let a non-technical user upload files, ask questions in natural language, and receive downloadable results from a simple web page

2. WHAT IS EXCEL AI ANALYZER?

Excel AI Analyzer is a web application that lets users upload multiple Excel or CSV files, ask questions in plain English (typed or spoken), and receive AI-generated analysis results that can be previewed and downloaded as Excel files.

Instead of manually writing formulas, creating pivot tables, or writing Python scripts, Excel AI Analyzer:

Reads uploaded files — Parses .xlsx, .xls, and .csv files into memory

Builds a data summary — Extracts shape, columns, data types, and sample rows

Sends to AI APIs — Routes the query through Gemini, Groq, or OpenRouter with automatic fallback if one API fails

Returns structured results — Delivers analysis summary, insights, and per-file result tables in a standard JSON format

Example

Without Excel AI Analyzer: A sales manager opens five Excel files, writes VLOOKUP formulas, creates pivot tables, and manually filters records to find top customers by revenue. The task takes an hour and is prone to formula errors.

With Excel AI Analyzer: The manager drags the five files into the web app, types "Show me top 10 customers by revenue," and receives a formatted table with insights in under 10 seconds. The result is downloaded as a new Excel file ready for the board presentation.

3. WHY IS EXCEL AI ANALYZER NECESSARY?

Current Challenges

Teams that work with Excel and CSV data routinely face:

Manual effort Significant time is lost writing formulas, creating pivot tables, and manually filtering data across multiple files

Human error Missed rows, wrong formulas, incorrect cell references, and inconsistent analysis across team members

Lack of scale As data volume grows, manual analysis becomes impractical. Multi-file analysis requires scripting knowledge that business users do not have

Skill barrier Business users without coding or advanced Excel skills cannot perform complex multi-file analysis

Poor tracking No simple record of which queries were run, what filters were applied, and what results were produced

4. BUSINESS PROBLEM STATEMENT

Existing Situation

Operations, sales, finance, and logistics teams regularly need to analyse data stored in Excel and CSV files, including:

- Sales reports and revenue summaries
- Customer databases and segmentation lists
- Inventory and stock-level spreadsheets
- Financial statements and transaction logs
- Shipping records and delivery manifests

Analysing these files is currently a manual, repetitive process that consumes valuable working hours and requires specialised skills.

Business Impact

- Increased operational cost from time spent on low-value data wrangling
- Reduced staff productivity as skilled analysts are tied up in routine tasks
- Delays in decision-making because insights are not extracted quickly enough
- Inconsistent analysis as different team members apply different methods
- Risk of errors in business-critical reports due to manual formula mistakes

Proposed Solution

Deploy Excel AI Analyzer: a lightweight web tool that lets users upload multiple spreadsheet files, ask questions in plain English, and receive structured, downloadable analysis results automatically while producing an audit-ready record of every query.

5. SOLUTION OVERVIEW AND KEY FEATURES

Excel AI Analyzer offers a single, unified workflow with multiple AI back-ends and a built-in local fallback engine.

Core Workflow

- 1. Upload Files** — The user drags and drops or selects multiple Excel (.xlsx, .xls) and CSV files. Files are parsed in memory and a schema summary (columns, types, shape, sample rows) is generated for each.
- 2. Configure API Keys** — The user pastes one or more API keys for Gemini, Groq, and/or OpenRouter. Keys are stored in browser localStorage and sent with each query. No keys are stored on the server.
- 3. Ask a Question** — The user types or speaks a natural-language query, such as: "Show me top 10 products by revenue", "Filter premium customers from Mumbai", "Revenue by category last quarter", or "Which city has the most orders?"
- 4. AI Analysis with Auto-Fallback** — The app attempts the query through each configured API in order: Gemini first, then Groq, then OpenRouter. If an API returns a quota or rate-limit error, the app automatically tries the next.
- 5. View and Download Results** — Results are displayed as structured tables with per-file insights. The user can download any result table as a new Excel file.

Shared Capabilities

- Multi-file upload** — Upload unlimited Excel and CSV files simultaneously (up to 50 MB total)
- Multi-API support** — Gemini (Google), Groq (Llama), OpenRouter (multiple free models) with automatic fallback chain
- Voice queries** — Built-in browser speech-to-text for hands-free querying
- File preview** — Preview uploaded file data (first 20 rows, columns, types, shape) before running analysis
- Structured results** — Every response includes analysis_summary, insights, and per-file tables with result_data, columns, total_rows, and table_insight
- Excel export** — Download any result table as a .xlsx file
- Local fallback** — If all AI APIs fail or no keys are provided, the app falls back to a local Pandas engine for basic filtering, sorting, and top-N extraction
- Dark theme UI** — Modern, professional dark-mode interface optimised for extended use

6. TECHNOLOGY STACK EVALUATION AND RECOMMENDATION

Three approaches were considered for analysing spreadsheet data.

Option 1 — Manual Analysis (Baseline): Staff continue to open files, write formulas, and create pivot tables by hand. No technology is required, but the approach does not scale and offers no record of what was analysed.

Option 2 — Excel AI Analyzer Web App (Recommended): Technology stack: Python 3.12, Flask, Pandas, OpenPyXL, Requests, google-generativeai, deployed locally or on any web host.

Advantages:

- Low implementation cost and minimal infrastructure (a single Python web service)
- Fast responses even on large files thanks to in-memory Pandas processing and parallel API calls
- Flexible natural-language querying — no formula knowledge needed
- Built-in structured JSON reporting for an audit trail
- Usable by non-technical staff through a simple web page
- Multiple free AI tiers mean near-zero running cost
- Local fallback ensures the app works even without internet or API keys

Option 3 — Enterprise BI Platform: Technology stack: Power BI, Tableau, or similar enterprise BI tool with ETL pipelines, data warehouses, and licenced seats. Highly capable and scalable, but carries higher cost, greater technical complexity, and a longer setup time — more than current needs require.

Comparison

Criteria	Manual analysis	Excel AI Analyzer (recommended)	Enterprise BI
Implementation cost	None	Low	High
Setup effort	None	Low	High
Speed on large files	Very slow	Fast	Fast
Query flexibility	Manual only	High (natural language)	High
Audit trail / reporting	None	JSON + Excel export	Dashboards
Infrastructure requirement	None	Minimal (1 service)	Significant
Technical complexity	Low	Low	High
Scalability	Very low	Medium	Very high
Business-user accessibility	Low	High	Low

Final Recommendation

Recommended Solution: Excel AI Analyzer — the Flask + Multi-API web app

Justification:

- It directly solves the current problem of analysing spreadsheet data without coding or advanced Excel skills
- It requires minimal infrastructure and is inexpensive to run
- It is fast on large files and flexible in what it analyses
- It produces a clear, structured record of every query and result
- It can be operated by business users with no coding knowledge
- It works offline via the local fallback engine
- It leaves room to grow into a fuller analytics platform later

7. HOW EXCEL AI ANALYZER WORKS (ARCHITECTURE)

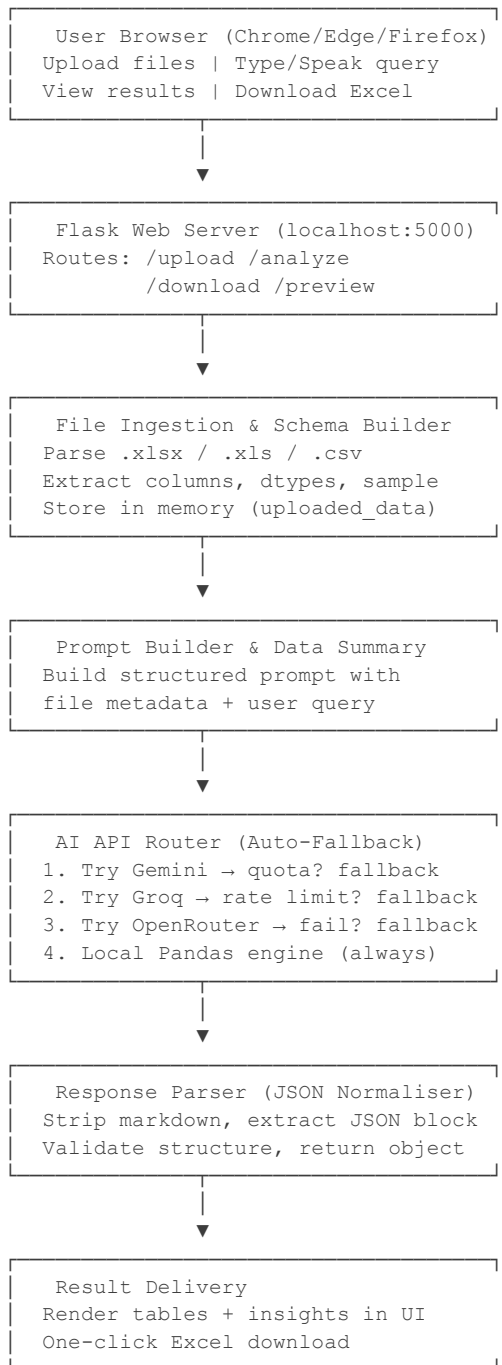


Figure 1 — Excel AI Analyzer data flow

The key idea: structured prompts, robust parsing, resilient fallback

Earlier approaches required users to write code or complex formulas to analyse data. On a dataset with thousands of rows across multiple files, this meant hours of manual work or hiring a data analyst.

Excel AI Analyzer instead works in four phases:

Phase 1 — File Ingestion — The app parses each uploaded file using Pandas, extracting the full dataframe into memory. It records the shape (rows x columns), column names, data types, and the first 10 rows as a text summary.

Phase 2 — Prompt Construction — A structured prompt is built combining the data summary, the user's natural-language query, and strict JSON formatting instructions. This prompt is sent to the AI API with a system message enforcing valid JSON output only.

Phase 3 — AI Routing with Fallback — The app attempts the query through each configured API in sequence: Gemini (gemini-2.0-flash) — 60 requests/min free tier; Groq (llama-3.3-70b-versatile) — very generous free tier; OpenRouter (llama-3.1-70b, gemma-2-9b, hermes-3-405b) — multiple free models with rotation. If any API returns a quota exceeded (429) or other error, the app automatically falls back to the next API in the chain.

Phase 4 — Local Fallback — If all AI APIs fail or no keys are configured, the app falls back to a local Pandas engine that performs basic operations: keyword-based filtering (WHERE / FILTER queries), numeric column sorting (TOP N queries), column-aware heuristic matching (revenue → sales amount columns), and safe defaults (first N rows) when heuristics do not match.

Only the analysis results are transferred back to the browser; raw file data never leaves the server, keeping runs fast and private.

8. SECURITY AND DEPLOYMENT

Security

API key handling — API keys are entered in the browser UI and stored in browser localStorage only. They are sent with each /analyze request but are never logged or stored on the server. Keys can be revoked at any time from the respective provider dashboards.

No persistent storage — Uploaded files are parsed into memory and the temporary disk copy is deleted immediately after parsing. No file data persists on the server between sessions.

Read-only analysis — The app only reads uploaded files. It never modifies, overwrites, or deletes source data.

Secure file handling — Uploaded filenames are sanitised using Werkzeug's secure_filename() to prevent path traversal attacks. File size is limited to 50 MB total per upload.

Deployment

The app runs as a single Flask web service on localhost:5000 by default. It can be deployed to any Python-capable host (Render, Heroku, AWS Elastic Beanstalk, etc.) with minimal configuration.

Auto-start scripts are provided for all platforms:

- Windows: double-click start.bat
- Linux/Mac: ./start.sh (creates venv, installs deps, launches)
- Any OS: python start.py (system Python, no venv)

Limitations

- Designed for single-user or small-team use; it is a focused personal/team tool, not a multi-tenant product.
- In-memory storage means uploaded data is lost when the server restarts. For persistent multi-user deployment, a database backend would be needed.
- The local fallback engine has limited intelligence compared to full AI analysis. Complex queries (multi-step reasoning, statistical analysis) require at least one AI API key.

9. API REFERENCE AND ENDPOINTS

The Flask backend exposes the following REST endpoints:

GET /

Renders the main UI template (templates/index.html)

POST /upload

Accepts multipart/form-data with one or more files in the 'files' field. Parses each file and returns schema metadata.

Request: multipart/form-data
files[] – Excel or CSV files

Response: JSON

```
{
  "success": true,
  "message": "3 file(s) uploaded successfully",
  "files": ["sales.xlsx", "customers.csv", "orders.xlsx"],
  "schema": { ... },
  "errors": null
}
```

POST /analyze

Accepts a JSON payload with the user query and API keys. Routes to AI APIs with auto-fallback and returns structured results.

Request: application/json

```
{
  "query": "top 10 customers by revenue",
  "gemini_key": "...",
  "groq_key": "...",
  "openrouter_key": "..."
}
```

Response: JSON

```
{
  "analysis_summary": "Brief summary...",
  "insights": "Key insights...",
  "tables": {
    "sales.xlsx": {
      "result_data": [...],
      "columns": [...],
      "total_rows": 10,
      "table_insight": "..."
    }
  },
  "api_used": "groq"
}
```

POST /download

Accepts a JSON payload with result data and returns an Excel file.

Request: application/json

```
{
  "result_data": [...],
  "columns": [...],
}
```

```
    "filename": "top_customers.xlsx"
}
```

Response: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
Excel file attachment

POST /preview

Returns a preview of an uploaded file (first 20 rows, columns, shape, data types).

Request: application/json
{ "filename": "sales.xlsx" }

Response: JSON

```
{
  "columns": ["Date", "Customer", "Revenue"],
  "preview": [...],
  "shape": [1000, 3],
  "dtypes": { ... }
}
```

